

Modul Praktikum

Algoritma dan Struktur Data (TPL2106)

Oleh:

Dr. Sofiyanti Indriasari, S.Kom., M.Kom.

**Teknologi Rekayasa Perangkat Lunak
Semester Genap
Tahun Akademik 2025/2026**

Modul 8

Graph II: Shortest Path (Dijkstra & Bellman-Ford)

1. Deskripsi Singkat Modul

Modul 8 membahas algoritma shortest path pada graph, yaitu algoritma yang digunakan untuk mencari jalur dengan total bobot terkecil dari satu node ke node lainnya. Konsep shortest path banyak digunakan dalam berbagai bidang, seperti navigasi peta digital, jaringan komputer, sistem transportasi, routing data, dan optimasi jalur distribusi.

Pada modul ini, mahasiswa akan mempelajari konsep weighted graph (graph berbobot), representasi bobot pada edge, serta implementasi dua algoritma shortest path yang paling umum digunakan, yaitu **Dijkstra Algorithm** dan **Bellman-Ford Algorithm**.

Mahasiswa akan memahami bagaimana menghitung jarak minimum antar node, membandingkan karakteristik kedua algoritma, serta mengimplementasikan shortest path menggunakan Python melalui latihan bertahap dan studi kasus sederhana.

2. Capaian Pembelajaran

Setelah mengikuti praktikum pada Modul 8 ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep shortest path pada graph.
2. Memahami konsep weighted graph.
3. Merepresentasikan graph berbobot dalam Python.
4. Mengimplementasikan algoritma Dijkstra.
5. Mengimplementasikan algoritma Bellman Ford.
6. Membandingkan karakteristik Dijkstra dan Bellman Ford.
7. Menentukan jalur dengan total bobot minimum.

8. Menulis kode program shortest path secara terstruktur dan terdokumentasi. Menulis kode program Graph secara terstruktur dan terdokumentasi.

3. Prasyarat Pengetahuan Awal

Sebelum mengikuti praktikum pada modul ini, mahasiswa diharapkan telah memahami:

- Konsep Graph
- Representasi adjacency list
- BFS dan DFS
- Queue dan Priority Queue dasar
- Dictionary dan List pada Python
- Perulangan dan percabangan
- Fungsi dalam Python

4. Kebutuhan Alat

A. Perangkat Keras

Laptop/PC dengan spesifikasi minimum:

- RAM minimal 4 GB (disarankan 8 GB)
- Ruang kosong minimal 1 GB

Koneksi internet (opsional) untuk instalasi awal dan update extension.

B. Perangkat Lunak

- Python 3.x (disarankan Python 3.10+)
- Visual Studio Code (VS Code)
- Extension VS Code: Python (Microsoft)
- Windows Command Prompt / PowerShell / Git Bash

C. Folder kerja praktikum

5. Intruksi Ketentuan Teknis Praktikum

- a) Setiap program ditulis dalam file Python sesuai nama yang ditentukan.

- b) Kode program harus diberi komentar pada bagian penting.
- c) Mahasiswa wajib menjalankan dan menguji program sebelum dikumpulkan.
- d) Program harus dapat dijalankan tanpa error.
- e) Mahasiswa harus memahami Algoritma Dijkstra dan Algoritma Bellman Ford, bukan hanya menyalin kode.
- f) Mahasiswa wajib menyimpan seluruh file praktikum dalam repository Github masing-masing sesuai dengan ketentuan yang berlaku.
- g) Penamaan file mengikuti instruksi modul

Praktikum12.materi1.py s/d praktikum12.Materi2.py

Praktikum12.latihan1.py s/d praktikum12.latihan5.py

- h) Setiap kode program harus diberi **komentar** untuk menjelaskan tujuan blok program sebagai pembiasaan dokumentasi kode. Serta gunakan komentar pada program terkait untuk menjawab pertanyaan-pertanyaan pada latihan

6. Materi

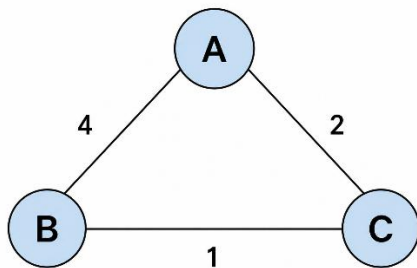
6.1 Konsep Weighted Graph

Pada modul sebelumnya, graph digunakan untuk merepresentasikan hubungan antar node tanpa memperhatikan nilai atau biaya perjalanan antar node tersebut. Pada kondisi tersebut, semua hubungan dianggap memiliki nilai yang sama.

Namun, dalam banyak kasus nyata, hubungan antar node memiliki **bobot (weight)** yang berbeda-beda. Bobot ini dapat merepresentasikan berbagai hal, seperti:

- jarak antar lokasi,
- biaya perjalanan,
- waktu tempuh,
- kapasitas jaringan,
- atau tingkat kemacetan.

Graph yang memiliki nilai pada setiap edge disebut sebagai **Weighted Graph**. Dengan adanya bobot, proses penelusuran graph menjadi lebih kompleks karena sistem tidak hanya mencari jalur yang tersedia, tetapi juga harus menentukan jalur dengan total bobot paling kecil atau paling optimal.



Graph di atas menunjukkan:

- Jalur dari A ke B memiliki bobot 4
- Jalur dari A ke C memiliki bobot 2
- Jalur dari B ke C memiliki bobot 1

Artinya, setiap perpindahan antar node memiliki “biaya” tertentu.

Sebagai contoh:

- Jika kita bergerak dari A ke B langsung, total biaya = 4
- Jika bergerak dari A → C → B, maka total biaya = 2 + 1 = 3

Dari contoh tersebut dapat dilihat bahwa jalur dengan edge paling sedikit belum tentu merupakan jalur terbaik. Pada graph biasa fokus utama adalah “apakah node terhubung?”. Sedangkan pada weighted graph fokus utama adalah **“berapa biaya untuk mencapai node tersebut?”**

Pada Python, weighted graph dapat direpresentasikan menggunakan dictionary bersarang (*nested dictionary*).

Contoh

```
graph = {
    'A': {'B': 4, 'C': 2},
    'B': {'C': 1},
    'C': {}
}
```

Cara Membaca : 'A': {'B': 4, 'C': 2}

Artinya:

- A terhubung ke B dengan bobot 4
- A terhubung ke C dengan bobot 2

Jika adjacency list biasa hanya menyimpan neighbor, maka weighted graph juga harus menyimpan bobot hubungan tersebut.

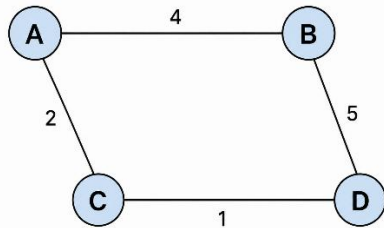
6.2 Konsep Shortest Path

Shortest path adalah proses mencari jalur dengan total bobot paling kecil dari satu node ke node lainnya. Konsep shortest path sangat penting dalam berbagai bidang, seperti:

- Google Maps,
- sistem navigasi kendaraan,
- routing jaringan internet,
- optimasi distribusi barang,

- dan sistem rekomendasi jalur transportasi.

Misalkan terdapat graph berikut : Berapa jalur terpendek dari A ke D?



Kemungkinan Jalur:

- Jalur 1 : A → B → D
Total Bobot = 4 + 5 = 9
- Jalur 2: A → C → D
Total Bobot = 2 + 1 = 3

Walaupun jalur sama - sama terdiri dari dua edge, jalur kedua memiliki total bobot lebih kecil. Maka jalur terpendek adalah A → C → D

Bagaimana jika jumlah edge lebih banyak tapi bobot lebih kecil daripada yang jumlah edgenya lebih sedikit? . Pada algoritma shortest path, ***jalur terbaik tidak ditentukan berdasarkan jumlah edge atau banyaknya node yang dilalui, melainkan berdasarkan total bobot terkecil dari seluruh jalur tersebut.*** Oleh karena itu, sebuah jalur yang memiliki edge lebih banyak tetap dapat menjadi jalur terpendek apabila total bobotnya lebih kecil dibandingkan jalur lain yang memiliki edge lebih sedikit. Sebagai contoh, jalur dengan 3 edge dan total bobot 4 akan lebih dipilih dibandingkan jalur dengan 2 edge tetapi total bobot 9, karena algoritma shortest path berfokus pada biaya minimum, bukan jumlah langkah perjalanan.

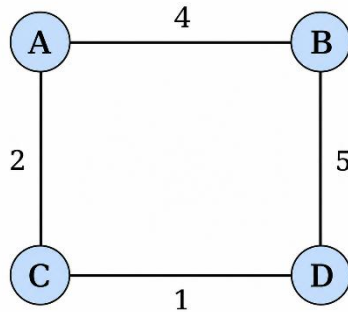
6.3 Algoritma Dijkstra

Algoritma Dijkstra digunakan untuk mencari jalur terpendek dari satu node ke seluruh node lain pada weighted graph dengan bobot positif. Algoritma ini bekerja dengan prinsip: “Selalu memilih node dengan jarak sementara paling kecil.”

Cara Kerja Dijkstra

Secara umum, langkah algoritma Dijkstra adalah:

1. Tentukan node awal
2. Beri jarak awal:
 - node awal = 0
 - node lain = tak hingga (∞)
3. Pilih node dengan jarak terkecil
4. Perbarui jarak tetangganya
5. Tandai node sebagai selesai diproses
6. Ulangi sampai semua node selesai



Langkah 1 Awal: Mulai dari Node A

Node	Jarak
A	0
B	∞
C	∞
D	∞

Proses Update

Dari A :

- Ke B = 4
- Ke C = 2

Node	Jarak
A	0
B	4
C	2
D	∞

Kemudian pilih node dengan jarak terkecil berikutnya, yaitu C.

Dijkstra bekerja seperti “mencari jalur termurah secara bertahap”

Implementasi Dijkstra dalam Python. Buat kode program di bawah ini dan simpan dengan nama **praktikum12.materi1.py**. Tambahkan penjelasan yang diperlukan dalam bentuk komentar dalam kode program.

```
import heapq
graph = {
    'A': {'B': 4, 'C': 2},
    'B': {'D': 5},
    'C': {'D': 1},
    'D': {}
}
```

```
def dijkstra(graph, start):
    # Menyimpan jarak minimum
    distances = {node: float('inf') for node in graph}

    # Jarak node awal = 0
    distances[start] = 0

    # Priority queue
    pq = [(0, start)]

    while pq:
        current_distance, current_node = heapq.heappop(pq)

        # Periksa semua tetangga
        for neighbor, weight in graph[current_node].items():

            distance = current_distance + weight

            # Jika ditemukan jarak lebih kecil
            if distance < distances[neighbor]:

                distances[neighbor] = distance

                heapq.heappush(pq, (distance, neighbor))

    return distances

hasil = dijkstra(graph, 'A')
print(hasil)
```

Output : {'A': 0, 'B': 4, 'C': 2, 'D': 3}

Penjelasan :

- Jarak dari A ke A = 0
- Jarak dari A ke B = 4
- Jarak dari A ke C = 2
- Jarak dari A ke D = 3

Karena A → C → D = 2 + 1 = 3

Kelemahan utama algoritma Dijkstra adalah algoritma ini tidak dapat bekerja dengan benar pada graph yang memiliki bobot negatif. Dijkstra menggunakan pendekatan greedy dengan asumsi bahwa jarak terpendek yang sudah dipilih tidak akan berubah lagi, sehingga jika terdapat edge dengan bobot negatif, algoritma dapat menghasilkan perhitungan shortest path yang salah. Selain itu, pada graph yang sangat besar dengan jumlah node dan edge yang

banyak, proses perhitungan juga dapat menjadi lebih kompleks dan membutuhkan penggunaan struktur data tambahan seperti priority queue agar tetap efisien.

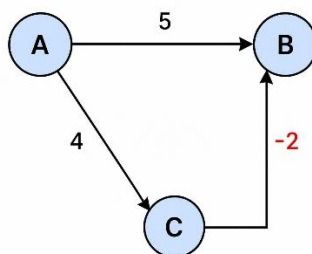
Contoh kelemahan algoritma Dijkstra dapat dilihat pada graph yang memiliki bobot negatif. Misalnya, terdapat jalur

- A → B dengan bobot 5,
- A → C dengan bobot 4,
- C → B dengan bobot -3.

Secara logika, jalur terpendek dari A ke B seharusnya melalui C, yaitu $4 + (-3) = 1$. Namun, algoritma Dijkstra dapat langsung memilih jalur A → B dengan bobot 5 lebih awal dan menganggapnya sebagai jalur terbaik, sehingga hasil shortest path menjadi tidak akurat.

6.4 Algoritma Bellman Ford

Bellman-Ford adalah algoritma shortest path yang dapat menangani bobot negatif. Berbeda dengan Dijkstra, Bellman-Ford bekerja dengan cara: “melakukan relaksasi seluruh edge secara berulang”. Perhatikan ilustrasi berikut ini



- A → B = 5
- A → C = 4
- C → B = -2

Jalur Terbaik :

- Langsung A → B = 5
- Melalui C maka: A → C → B = $4 + (-2) = 2$

Pada ilustrasi Bellman-Ford di atas, terdapat tiga jalur hubungan antar node, yaitu A → B dengan bobot 5, A → C dengan bobot 4, dan C → B dengan bobot -2. Jika dilihat secara langsung, jalur dari A ke B memiliki total bobot 5. Namun, ketika melalui node C, total bobot menjadi lebih kecil, yaitu $4 + (-2) = 2$. Hal ini menunjukkan bahwa meskipun jalur melalui lebih banyak edge, total biaya perjalanan dapat menjadi lebih kecil karena adanya bobot negatif. Kondisi seperti ini merupakan salah satu alasan mengapa algoritma Bellman Ford digunakan, karena algoritma ini mampu menangani graph dengan bobot negatif dan tetap menemukan jalur terpendek secara benar, berbeda dengan Dijkstra yang dapat menghasilkan hasil tidak akurat pada kasus tersebut.

Implementasi Bellman Ford pada Python. Buat kode program di bawah ini dan simpan dengan nama **praktikum12.materi2.py**. Tambahkan penjelasan yang diperlukan dalam bentuk komentar dalam kode program.

```
def bellman_ford(graph, start):  
  
    distances = {node: float('inf') for node in graph}  
    distances[start] = 0  
  
    # Relaksasi berulang  
    for _ in range(len(graph) - 1):  
  
        for node in graph:  
  
            for neighbor, weight in graph[node].items():  
  
                if distances[node] + weight < distances[neighbor]:  
  
                    distances[neighbor] = distances[node] + weight  
  
    return distances
```

Algoritma Bellman-Ford merupakan algoritma shortest path yang mampu menangani graph dengan bobot negatif. Algoritma ini bekerja dengan melakukan relaksasi seluruh edge secara berulang untuk memastikan setiap node memperoleh jarak minimum yang benar. Meskipun prosesnya lebih lambat dibandingkan Dijkstra, Bellman-Ford memiliki keunggulan dalam mendeteksi dan menangani edge berbobot negatif, sehingga lebih fleksibel digunakan pada kasus tertentu. Dengan memahami Bellman-Ford, mahasiswa dapat membandingkan pendekatan shortest path yang berbeda serta memahami bahwa pemilihan algoritma harus disesuaikan dengan karakteristik graph yang digunakan.

Perbandingan Dijkstra dan Bellman Ford

Aspek	Dijkstra	Bellman-Ford
Bobot negatif	Tidak bisa	Bisa
Kecepatan	Lebih cepat	Lebih lambat
Pendekatan	Greedy	Relaksasi

7. Latihan - Latihan

Bagian ini merupakan inti kegiatan praktikum. Mahasiswa diminta membaca ulang materi pada Bagian 6, menjalankan kode program yang tersedia, mengamati output, menjawab pertanyaan analisis dalam bentuk komentar pada file program, serta mengembangkan program sesuai instruksi soal. Setiap latihan disimpan dalam file Python terpisah dengan format penamaan:

- praktikum12.latihan1.py
- praktikum12.latihan2.py
- praktikum12.latihan3.py
- praktikum12.latihan4.py

Setiap file wajib diberi header identitas mahasiswa di bagian atas program:

```
# Nama :  
# NIM :  
# Kelas :  
# Praktikum 12 - Graph II: Shortest Path
```

Mahasiswa wajib menuliskan jawaban analisis sebagai komentar di bagian bawah file program.

Latihan 1. Memahami Weighted Graph dan Shortest Path Sederhana

Salin, jalankan, dan amati kode berikut. Setelah program dijalankan, jawab pertanyaan analisis dalam bentuk komentar pada file program.

```
# =====  
# Latihan 1: Weighted Graph dan Perhitungan Jalur  
# =====  
  
# Representasi weighted graph menggunakan dictionary bersarang  
graph = {  
    'A': {'B': 4, 'C': 2},  
    'B': {'D': 5},  
    'C': {'D': 1},  
    'D': {}  
}  
  
# Menghitung dua kemungkinan jalur dari A ke D  
jalur_1 = graph['A']['B'] + graph['B']['D'] # A -> B -> D  
jalur_2 = graph['A']['C'] + graph['C']['D'] # A -> C -> D  
  
print("Jalur 1: A -> B -> D =", jalur_1)  
print("Jalur 2: A -> C -> D =", jalur_2)
```

```
if jalur_1 < jalur_2:
    print("Jalur terpendek adalah A -> B -> D")
else:
    print("Jalur terpendek adalah A -> C -> D")
```

Pertanyaan Analisis

Tuliskan jawaban sebagai komentar di bagian bawah program.

```
# Jawaban Analisis:
# 1. Berapa total bobot jalur A -> B -> D?
# 2. Berapa total bobot jalur A -> C -> D?
# 3. Jalur mana yang dipilih sebagai jalur terpendek?
# 4. Mengapa jalur terpendek tidak selalu ditentukan dari jumlah edge yang
    paling sedikit?
```

Latihan 2. Mencoba Algoritma Dijkstra

Salin dan jalankan kode berikut. Amati hasil output yang muncul, kemudian jawab pertanyaan analisis dalam bentuk komentar pada file program.

```
# =====
# Latihan 2: Implementasi Dijkstra
# =====

import heapq

# Weighted graph dengan bobot positif
graph = {
    'A': {'B': 4, 'C': 2},
    'B': {'D': 5},
    'C': {'D': 1},
    'D': {}
}

def dijkstra(graph, start):
    """
    Fungsi untuk mencari jarak terpendek dari node start
    ke seluruh node lain menggunakan algoritma Dijkstra.
    """

    # Semua jarak awal dibuat tak hingga
    distances = {node: float('inf') for node in graph}

    # Jarak dari start ke start adalah 0
    distances[start] = 0

    # Priority queue menyimpan pasangan (jarak, node)
    priority_queue = [(0, start)]
```

```
while priority_queue:
    current_distance, current_node = heapq.heappop(priority_queue)

    # Jika jarak saat ini lebih besar dari jarak yang sudah tercatat,
    # maka proses dilewati
    if current_distance > distances[current_node]:
        continue

    # Periksa semua tetangga dari node saat ini
    for neighbor, weight in graph[current_node].items():
        distance = current_distance + weight

        # Jika ditemukan jarak yang lebih kecil, perbarui jaraknya
        if distance < distances[neighbor]:
            distances[neighbor] = distance
            heapq.heappush(priority_queue, (distance, neighbor))

return distances

hasil = dijkstra(graph, 'A')

print("Jarak terpendek dari node A:")
for node, distance in hasil.items():
    print(node, "=", distance)
```

Pertanyaan Analisis

Tuliskan jawaban sebagai komentar di bagian bawah program.

- ```
Jawaban Analisis:
1. Berapa jarak terpendek dari A ke B?
2. Berapa jarak terpendek dari A ke C?
3. Berapa jarak terpendek dari A ke D?
4. Mengapa jarak A ke D lebih kecil melalui C dibandingkan melalui B?
5. Apa fungsi priority_queue dalam algoritma Dijkstra?
6. Mengapa Dijkstra tidak cocok untuk graph dengan bobot negatif?
```

### Latihan 3. Mencoba Algoritma Bellman Ford

Salin dan jalankan kode berikut. Amati hasil output, kemudian jawab pertanyaan analisis dalam bentuk komentar pada file program.

```
=====
Latihan 3: Implementasi Bellman-Ford
=====

Weighted graph dengan bobot negatif
graph = {
 'A': {'B': 5, 'C': 4},
 'B': {},
 'C': {'B': -2}
}

def bellman_ford(graph, start):
 """
 Fungsi untuk mencari jarak terpendek dari node start
 ke seluruh node lain menggunakan algoritma Bellman-Ford.
 """

 # Semua jarak awal dibuat tak hingga
 distances = {node: float('inf') for node in graph}

 # Jarak dari start ke start adalah 0
 distances[start] = 0

 # Bellman-Ford melakukan relaksasi sebanyak jumlah node - 1
 for _ in range(len(graph) - 1):

 # Periksa semua edge
 for node in graph:
 for neighbor, weight in graph[node].items():

 # Jika jarak ke node saat ini sudah diketahui,
 # dan ditemukan jarak yang lebih kecil ke neighbor,
 # maka lakukan update jarak
 if distances[node] != float('inf') and distances[node] +
weight < distances[neighbor]:
 distances[neighbor] = distances[node] + weight

 return distances

hasil = bellman_ford(graph, 'A')

print("Jarak terpendek dari node A:")
for node, distance in hasil.items():
 print(node, "=", distance)
```

### Pertanyaan Analisis

Tuliskan jawaban sebagai komentar di bagian bawah program.

- # Jawaban Analisis:
- # 1. Berapa bobot langsung dari A ke B?
- # 2. Berapa total bobot jalur A -> C -> B?
- # 3. Jalur mana yang menghasilkan jarak lebih kecil menuju B?
- # 4. Mengapa Bellman-Ford dapat digunakan pada graph dengan bobot negatif?
- # 5. Apa yang dimaksud dengan proses relaksasi edge?
- # 6. Apa perbedaan utama Bellman-Ford dan Dijkstra?

### Latihan 4. Studi Kasus Contoh: Jalur Terpendek Antar Lokasi Kampus

Sebuah kampus memiliki beberapa lokasi yang saling terhubung. Bobot pada graph merepresentasikan perkiraan waktu tempuh dalam menit. Mahasiswa diminta menjalankan program untuk mengetahui waktu tempuh terpendek dari Gerbang Kampus ke lokasi lainnya.

```
=====
Latihan 4: Studi Kasus Jalur Terpendek Lokasi Kampus
Algoritma: Dijkstra
=====

import heapq

Graph lokasi kampus
Bobot menunjukkan waktu tempuh dalam menit
graph = {
 'Gerbang': {'Perpustakaan': 6, 'Kantin': 2},
 'Perpustakaan': {'Lab': 3},
 'Kantin': {'Lab': 4, 'Aula': 7},
 'Lab': {'Aula': 1},
 'Aula': {}
}

def dijkstra(graph, start):
 distances = {node: float('inf') for node in graph}
 distances[start] = 0

 priority_queue = [(0, start)]

 while priority_queue:
 current_distance, current_node = heapq.heappop(priority_queue)

 if current_distance > distances[current_node]:
 continue

 for neighbor, weight in graph[current_node].items():
```

```
distance = current_distance + weight

if distance < distances[neighbor]:
 distances[neighbor] = distance
 heapq.heappush(priority_queue, (distance, neighbor))

return distances

hasil = dijkstra(graph, 'Gerbang')

print("Jarak terpendek dari Gerbang Kampus:")
for lokasi, jarak in hasil.items():
 print(lokasi, "=", jarak, "menit")
```

### Pertanyaan Analisis

Tuliskan jawaban sebagai komentar di bagian bawah program.

```
Jawaban Analisis:
1. Lokasi mana yang paling dekat dari Gerbang?
2. Berapa waktu tempuh terpendek dari Gerbang ke Aula?
3. Apakah jalur langsung selalu menghasilkan jarak paling kecil? Jelaskan.
4. Mengapa Dijkstra cocok digunakan pada kasus lokasi kampus ini?
```

### Latihan 5. Studi Kasus dengan Program Shortest Path

Buatlah program untuk mencari jalur terpendek pada satu kasus berikut. Buat graph berbobot yang merepresentasikan hubungan antar kota berikut:

- Bogor → Jakarta = 5
- Bogor → Depok = 2
- Depok → Jakarta = 2
- Jakarta → Bandung = 7
- Depok → Bandung = 6

Gunakan algoritma Dijkstra untuk mencari jarak terpendek dari **Bogor** ke semua kota lainnya. Program harus memuat:

1. Representasi graph berbobot menggunakan dictionary.
2. Fungsi Dijkstra.
3. Input node awal atau minimal penentuan node awal dalam program.
4. Output jarak terpendek dari node awal ke semua node.
5. Komentar penjelasan pada bagian penting kode program.



### Output Yang Diharapkan

Jarak terpendek dari Bogor:

Bogor -> Bogor = 0

Bogor -> Jakarta = 4

Bogor -> Depok = 2

Bogor -> Bandung = 8

### Pertanyaan Analisis

Tuliskan jawaban sebagai komentar di bagian bawah file.

```
Jawaban Analisis:
1. Node awal yang digunakan apa?
2. Node mana yang memiliki jarak paling kecil dari node awal?
3. Node mana yang memiliki jarak paling besar dari node awal?
4. Jelaskan bagaimana algoritma Dijkstra bekerja pada kasus yang Anda buat.
```

### Ketentuan Praktikum

Buatlah semua program latihan pada bagian 6 dan bagian 7 dan masing-masing disimpan dalam file dengan nama

Program Bagian 6 Materi :

**Praktikum12.materi1.py s/d praktikum12.Materi2.py**

Program Bagian 7 Latihan

**Praktikum12.latihan1.py s/d praktikum12.latihan5.py**

Setiap file diberikan header identitas mahasiswa pada bagian atas kode program. Diskusikan kemudian tulis penjelasan dan hasil diskusi pada file program sebagai komentar. Semakin jelas dan lengkap akan semakin bagus. Kumpulkan di akhir praktikum di repo masing-masing pada **folder praktikum12**.